

# Code Explanation

## Batch Processing Pipeline Explanation

The provided Python script is a batch processing pipeline implemented using Apache Spark and Delta Lake. It reads customer and order data from CSV files, cleans and processes the data, and then saves it to Delta tables. The pipeline also creates and updates two derived tables: `ordersummary` with SCD Type 2 logic and `customeraggregatespend` with aggregated data.

### ## Overview of the Code

The code is structured into several functions, each responsible for a specific task in the data processing pipeline. The main function orchestrates the entire process, from initializing a Spark session to creating the final derived tables.

### ## Step 1: Creating a Spark Session

This step initializes a Spark session with the required configurations for Delta Lake.

```
def create_spark_session():
    """Create and return a Spark session."""
    return (SparkSession.builder
            .appName("Customer Order Processing")
            .config("spark.sql.extensions", "io.delta.sql.DeltaSpark
SessionExtension")
            .config("spark.sql.catalog.spark_catalog", "org.apache.s
park.sql.delta.catalog.DeltaCatalog")
            .getOrCreate())
```

### ## Step 2: Reading Source Data

This step reads customer and order data from specified CSV file paths.

```
def read_source_data(spark, customer_path, order_path):
    """Read source CSV data from specified paths."""
    customer_df = (spark.read.format("csv")
                   .option("header", "true")
                   .option("inferSchema", "true")
                   .load(customer_path))

    order_df = (spark.read.format("csv")
                .option("header", "true")
                .option("inferSchema", "true")
                .load(order_path))

    return customer_df, order_df
```

### ## Step 3: Cleaning Data

This step removes null values and duplicates from the customer and order data.

```
def clean_data(customer_df, order_df):
    """Clean data by removing nulls and duplicates."""
    # Clean customer data
    cleaned_customer_df = (customer_df
```

```

        .filter(
            (F.col("CustId").isNotNull()) &
            (F.col("Name").isNotNull()) &
            (F.col("EmailId").isNotNull()) &
            (F.col("Region").isNotNull())
        )
        .dropDuplicates(["CustId"]))

# Clean order data and add TotalAmount column
cleaned_order_df = (order_df
    .filter(
        (F.col("OrderId").isNotNull()) &
        (F.col("ItemName").isNotNull()) &
        (F.col("PricePerUnit").isNotNull()) &
        (F.col("Qty").isNotNull()) &
        (F.col("Date").isNotNull()) &
        (F.col("CustId").isNotNull())
    )
    .dropDuplicates(["OrderId"])
    .withColumn("TotalAmount", F.col("PricePerUnit") * F.col("Qty")))

return cleaned_customer_df, cleaned_order_df

```

## Step 4: Creating or Updating the `ordersummary` Table

This step creates or updates the `ordersummary` table with SCD Type 2 logic.

```

def create_or_update_ordersummary(spark, customer_df, order_df, catalog, schema):
    """Create or update the ordersummary SCD Type 2 table."""
    # Create catalog.schema if not exists
    spark.sql(f"CREATE CATALOG IF NOT EXISTS {catalog}")
    spark.sql(f"CREATE SCHEMA IF NOT EXISTS {catalog}.{schema}")

    # Join customer and order data
    joined_df = (customer_df
        .join(order_df, "CustId", "inner")
        .select(
            "CustId", "Name", "EmailId", "Region", "OrderId",
            "ItemName", "PricePerUnit", "Qty", "Date"
        ))

    # Check if ordersummary table exists
    table_exists = False
    try:
        spark.table(f"{catalog}.{schema}.ordersummary")
        table_exists = True
    except:
        table_exists = False

    if not table_exists:
        # First-time load with SCD Type 2 columns
        (joined_df
            .withColumn("IsActive", F.lit(True))
            .withColumn("StartDate", F.current_timestamp())

```

```

        .withColumn("EndDate", F.lit(None).cast("timestamp"))
        .write
        .format("delta")
        .mode("overwrite")
        .saveAsTable(f"{catalog}.{schema}.ordersummary"))
    else:
        # Implement SCD Type 2 logic for updates
        ordersummary_table = DeltaTable.forName(spark, f"{catalog}.{schema}.ordersummary")

        # Prepare data for merge
        update_data = (joined_df
            .withColumn("mergeKey", F.col("CustId"))
            .withColumn("IsActive", F.lit(True))
            .withColumn("StartDate", F.current_timestamp())
        )

        .withColumn("EndDate", F.lit(None).cast("timestamp")))

        # Perform the merge operation
        (ordersummary_table.alias("target")
            .merge(
                update_data.alias("source"),
                """target.CustId = source.CustId AND
                target.IsActive = true AND
                (target.Name != source.Name OR
                target.EmailId != source.EmailId OR
                target.Region != source.Region)"""
            )
            .whenMatchedUpdate(
                set={
                    "IsActive": "false",
                    "EndDate": "current_timestamp()"
                }
            )
            .whenNotMatchedInsertAll()
            .execute())

        # Insert new records for the updated customers
        new_records = (joined_df
            .join(
                spark.table(f"{catalog}.{schema}.ordersummary")
                    .filter(F.col("IsActive") == False)
                    .select("CustId")
                    .distinct(),
                "CustId",
                "inner"
            )
            .withColumn("IsActive", F.lit(True))
            .withColumn("StartDate", F.current_timestamp())
        )

        .withColumn("EndDate", F.lit(None).cast("timestamp")))

        if new_records.count() > 0:
            new_records.write.format("delta").mode("append").saveAsTable(f"{catalog}.{schema}.ordersummary")

```

## Step 5: Creating the `customeraggregatespend` Table

This step creates the `customeraggregatespend` table with aggregated data.

```
def create_customeraggregatespend(spark, catalog, schema):
    """Create the customeraggregatespend table with aggregated data.
    """
    # Create table if not exists
    spark.sql(f"""
        CREATE TABLE IF NOT EXISTS {catalog}.{schema}.customeraggregatespend (
            Name STRING,
            TotalAmount DOUBLE,
            Date DATE
        )
        USING DELTA
    """)

    # Calculate aggregations and update the table
    agg_df = (spark.table(f"{catalog}.{schema}.ordersummary")
              .join(
                  spark.table(f"{catalog}.{schema}.ordersummary")
                  .select("OrderId", F.col("PricePerUnit") * F.col("Quantity").alias("TotalAmount")),
                  "OrderId",
                  "inner"
              )
              .filter(F.col("IsActive") == True)
              .groupBy("Name", "Date")
              .agg(F.sum("TotalAmount").alias("TotalAmount")))

    # Write to the target table
    agg_df.write.format("delta").mode("overwrite").saveAsTable(f"{catalog}.{schema}.customeraggregatespend")
```

## Step 6: Main Function

This step orchestrates the entire data processing pipeline.

```
def main():
    """Main function to orchestrate the data processing pipeline."""
    # Initialize Spark session
    spark = create_spark_session()

    # Define paths and configurations
    customer_path = "/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/customerdata"
    order_path = "/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/orderdata"
    catalog = "gen_ai_poc_databrickscoe"
    schema = "sdlc_wizard"

    # Read source data
    customer_df, order_df = read_source_data(spark, customer_path, order_path)
```

```
# Clean data
cleaned_customer_df, cleaned_order_df = clean_data(customer_df,
order_df)

# Save cleaned data to Delta tables
cleaned_customer_df.write.format("delta").mode("overwrite").save
AsTable(f"{catalog}.{schema}.customer")
cleaned_order_df.write.format("delta").mode("overwrite").saveAsT
able(f"{catalog}.{schema}.order")

# Create or update ordersummary table (SCD Type 2)
create_or_update_ordersummary(spark, cleaned_customer_df, cleane
d_order_df, catalog, schema)

# Create customeraggregatespend table
create_customeraggregatespend(spark, catalog, schema)

spark.stop()

if __name__ == "__main__":
    main()
```